

Lezione 7: Threads & Concorrenza



Obiettivi

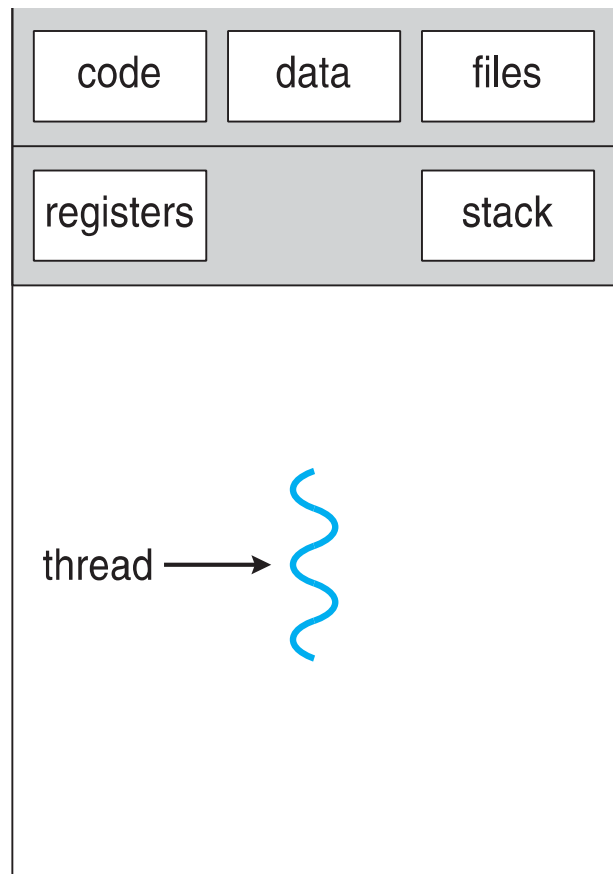
- Introdurre la nozione di thread
- Multithreaded programming
- Supporti per threads in Windows e Linux

Motivazioni

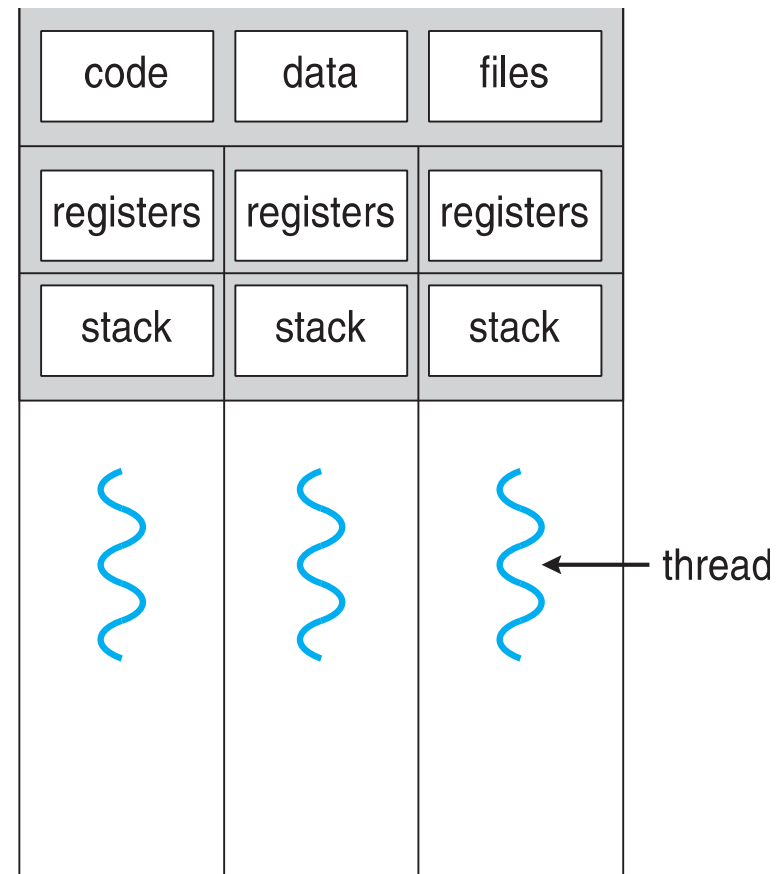
- Le applicazioni sono generalmente multithreaded
- Task multipli in applicazioni implementati da thread separati
 - Update del display
 - Fetch data
 - Spell checking
 - Risposta in rete
- La creazione di processo è pesante, il thread è leggero
- Il kernel è generalmente multithreaded

Processi Single e Multithreaded

- Unità base di utilizzo di CPU
 - Caratterizzati da Thread ID, PC, Registri e Stack
 - Minima identità di task



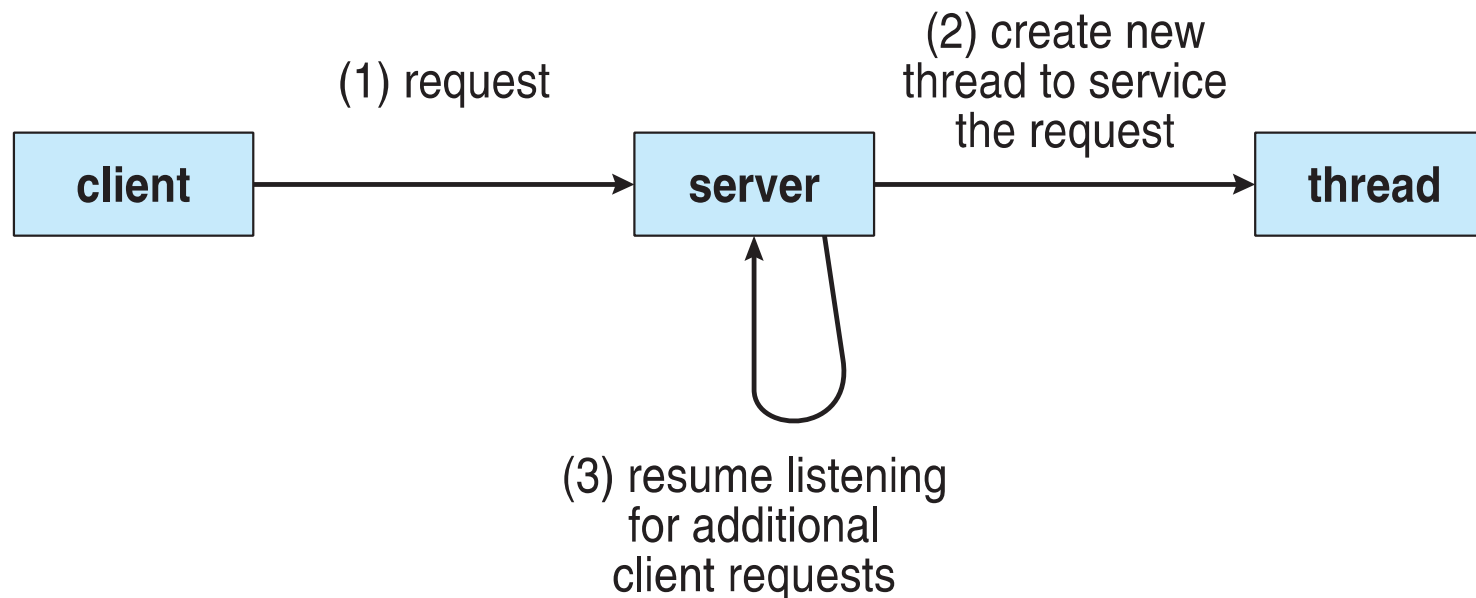
single-threaded process



multithreaded process

Architettura di Server Multithread

- Molto utilizzate nelle architetture client-server
 - Per ogni client si genera un thread che gestisce il client
 - Per ogni richiesta di un client un thread
 - Server può gestire tante richieste eseguendo molti task leggeri



Benefici

□ Risposta

- Esecuzione continua se una parte del processo è bloccata (importante per le interfacce utente)

□ Risorse

- Thread condividono le risorse di un processo, più semplice di shared memory o message passing

□ Economia

- Più leggero di una creazione di processo, il **thread switching** ha minore overhead del **context switching**

□ Scalabilità

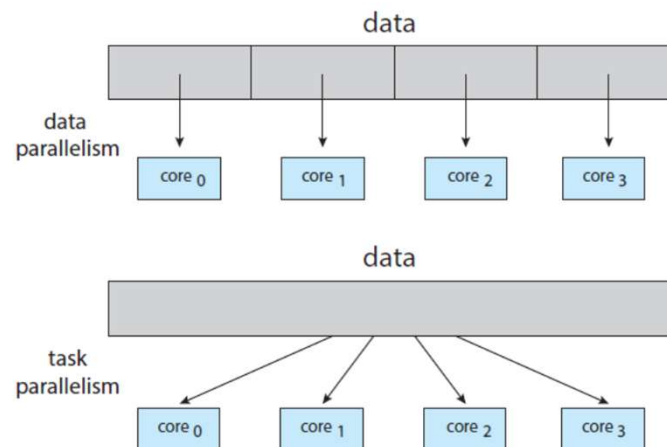
- I processi possono avvantaggiarsi delle architetture multiprocessore

Multicore Programming

- ❑ Sistemi multitread forniscono meccanismi utili per sistemi multicore
- ❑ Sistemi **Multicore** o **multiprocessori** pongono diversi problemi:
 - ❑ **Come dividere le attività sui core**
 - ▶ Computazioni parallelizzabili su più core
 - ❑ **Come bilanciare i calcoli sui core**
 - ❑ **Come splittare i dati**
 - ❑ **Come gestire le dipendenze dei dati sui core**
 - ▶ Dipendenze possono richiedere sincronizzazioni
 - ❑ **Come fare test e debugging**
 - ▶ Più tracce di esecuzioni da testare
- ❑ **Parallelismo**
 - ❑ Più di un task simultaneamente distribuiti su più unità di calcolo
- ❑ **Concorrenza**
 - ❑ Consente il progresso contemporaneo di più di un task
 - ❑ Se processore/core singolo, concorrenza data dallo scheduler che intralaccia le esecuzioni

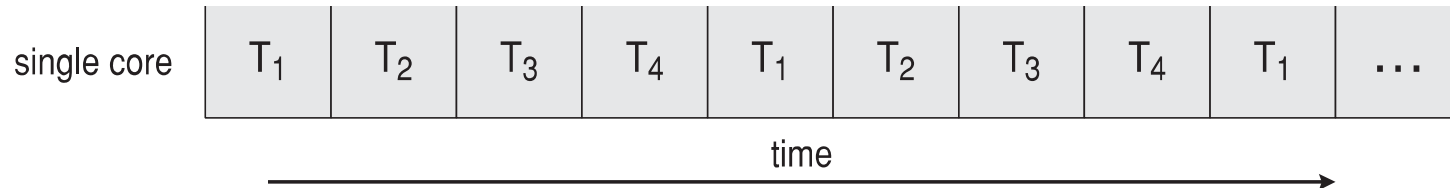
Multicore Programming

- Tipi di parallelismo
 - **Parallelismo di dati** – sottoinsiemi dei dati su core multipli con stessa operazione per ognuno
 - ▶ Ad esempio calcolo della somma dei numeri su array, si può dividere il compito
 - **Parallelismo di task** – thread sui core dove ciascuno svolge un'operazione particolare
 - ▶ Esempio media e varianza di un set di dati
- Al crescere del numero dei thread aumenta il supporto architetturale
 - CPU hanno core e **hardware thread**
 - Esempio, i7 (13th Gen) con 16 core e 24 hardware threads per core

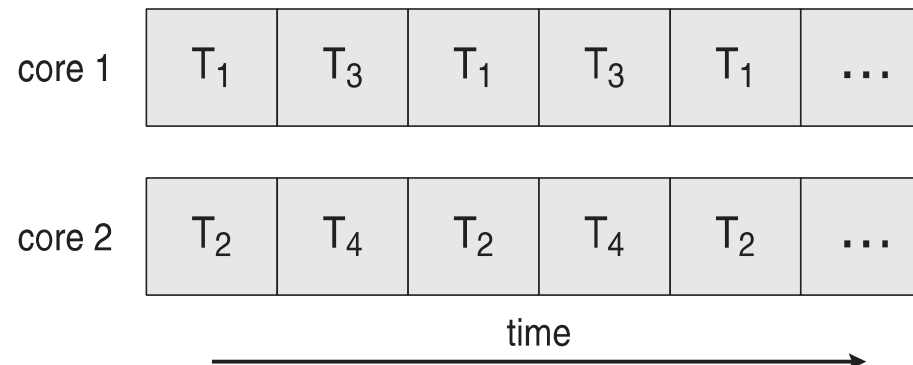


Concorrenza vs. Parallelismo

□ Esecuzione concorrente su sistema single-core:



□ Parallelismo su sistema multi-core:



Legge di Amdahl

- Guadagno teorico in performance con l'aggiunta di un core per un'applicazione che ha sia componenti seriali che paralleli
 - S è la porzione seriale
 - N sono i core per il processamento

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

speed-up rispetto ad un singolo core

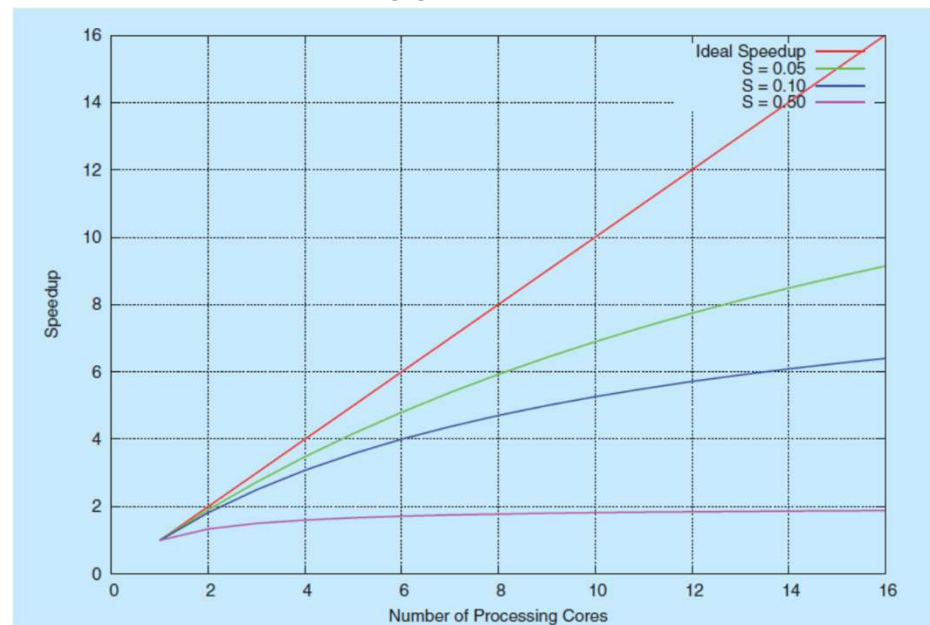
- Esempio: se l'applicazione è al 75% parallela e 25% seriale, passando da 1 a 2 core si ha uno speed-up di 1.6 volte
- Con N che tende ad infinito lo speed-up tende a $1 / S$

Legge di Amdahl

- Guadagno teorico in performance (latenza) con l'aggiunta di un core per un'applicazione che ha sia componenti seriali che paralleli
 - S è la porzione di calcolo seriale (non parallelizzabile)
 - N sono i core per il processamento

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- La porzione seriale di un'applicazione ha effetto molto marcato sul guadagno di performance con un core aggiuntivo



User Thread e Kernel Thread

- **User thread** – gestita da librerie di thread di livello utente

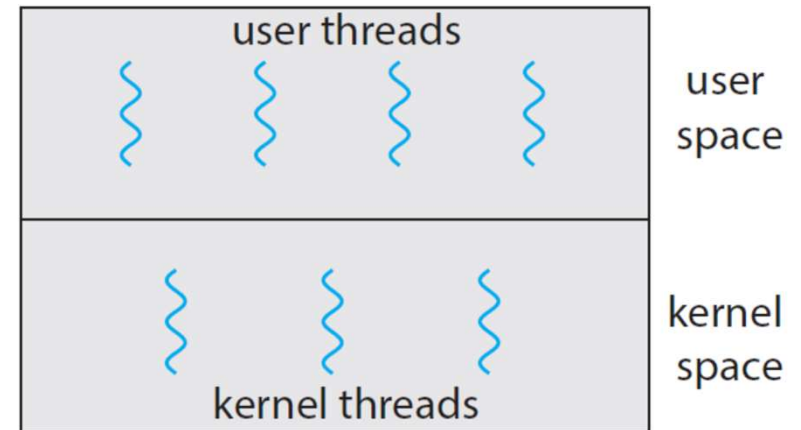
- Tre principali:

- POSIX **Pthreads**
- Windows threads
- Java threads

- **Kernel thread** - supportati dal Kernel

- Esempi – tutti gli SO general purpose inclusi:

- Windows
- Solaris
- Linux
- Tru64 UNIX
- Mac OS

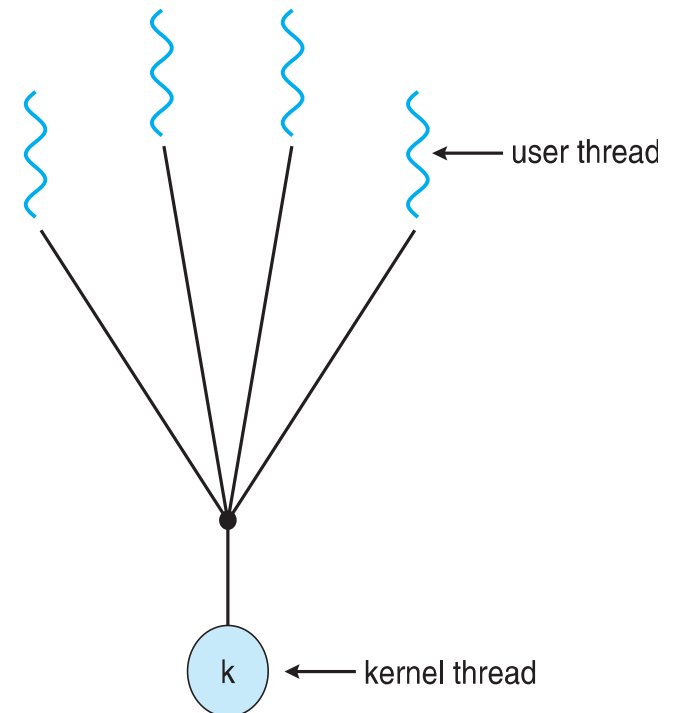


Modelli di Multithreading

- Diversi modi di gestire la relazione tra User e Kernel thread
 - Many-to-One
 - One-to-One
 - Many-to-Many

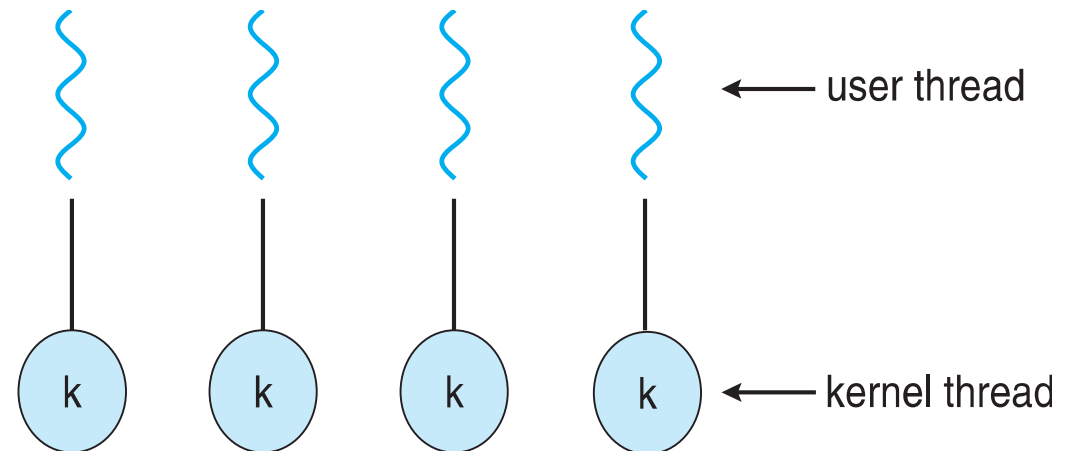
Many-to-One

- ❑ Tanti thread user mappati su un singolo kernel thread
- ❑ Però un thread bloccante (es. durante una system call) può bloccare tutto
- ❑ Problemi di parallelismo con sistemi multicore perché uno solo alla volta è in esecuzione nel Kernel
- ❑ Pochi sistemi utilizzano questo approccio:
 - ❑ **Solaris Green Threads** library
 - ❑ **GNU Portable Threads** library



One-to-One

- Ogni thread user-level mappato su kernel thread
 - La creazione di thread user-level crea un kernel thread
 - Maggiore concorrenza di many-to-one
 - Il numero di thread per processo può essere limitato per evitare overhead
 - Esempi:
 - ▶ Windows
 - ▶ Linux
 - ▶ Solaris 9 e successivi
- Utente deve controllare il numero dei thread



Many-to-Many

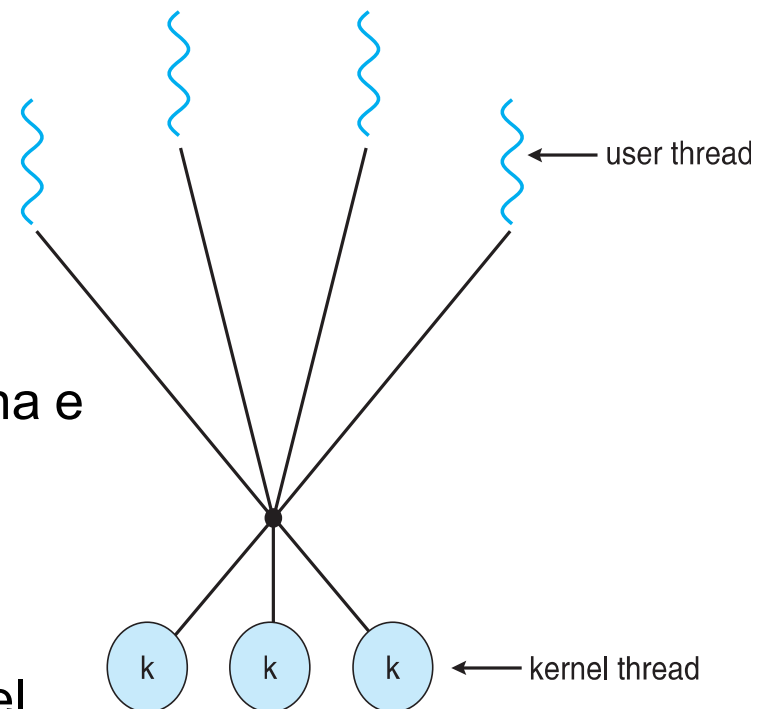
- ❑ Molti thread user-level sono mappati su molti thread del kernel
- ❑ Il SO può creare un numero sufficiente di thread del kernel
 - ❑ Però può gestirne il numero e il grado di parallelismo

- ❑ Esempi:

- ❑ Solaris dalla versione 9
 - ❑ Windows con *ThreadFiber* package

- ❑ Il numero può dipendere dalla macchina e dall'applicazione

- ❑ L'utente può lanciare più thread, poi controlla il Sistema quanti thread kernel

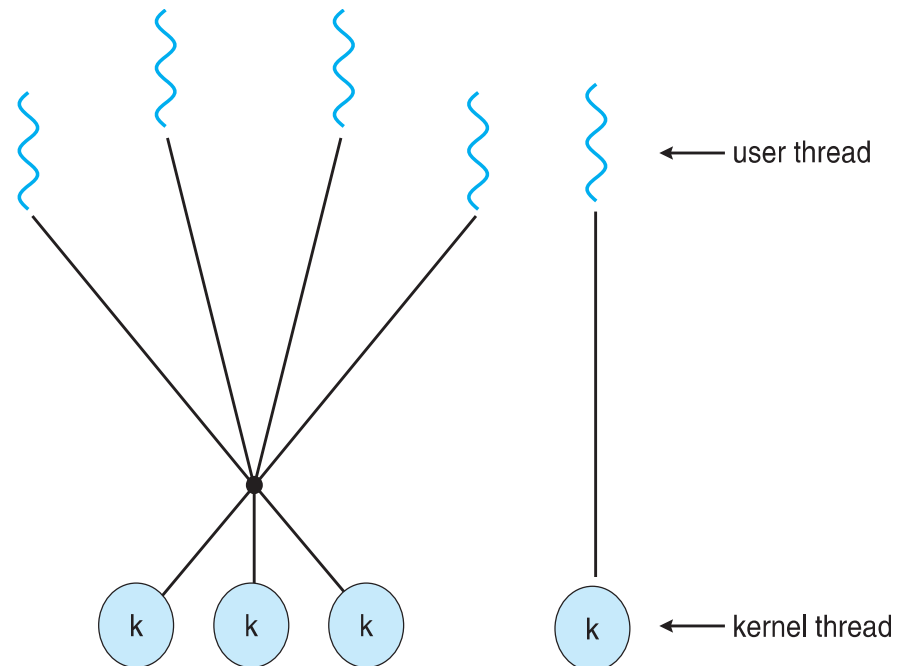


Two-level Model

- Variante del Many-to-Many che permette ad un user-thread di essere legato (**bound**) ad un kernel thread

- Esempi:

- IRIX
- HP-UX
- Tru64 UNIX
- Solaris 8 e precedenti

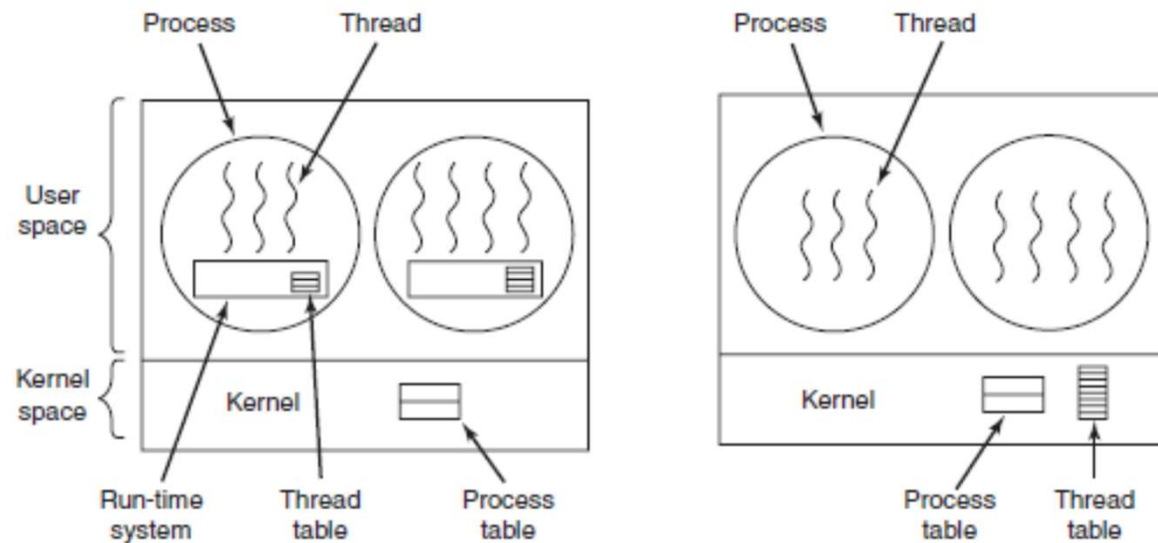


Librerie per Thread

- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread
- Due modi principali di implementarle
 - Libreria interamente nello user space
 - ▶ I thread sono implementati al livello utente (gestiti con chiamate di funzione utente)
 - Libreria al livello Kernel supportata dal SO
 - ▶ I thread sono gestiti con chiamate di Sistema (gestiti con chiamate di sistema)

Librerie per Thread

- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread
- Due modi principali di implementarle
 - Libreria interamente nello user space
 - Libreria al livello Kernel supportata dall'SO



Pthreads

- Estensione di POSIX standard (IEEE 1003.1c) fornisce le API per la creazione e sincronizzazione dei thread
- Esiste sia in versione user-level sia kernel-level
- **Specifica**, non **implementazione**
- API specifica il comportamento delle funzioni di libreria, implementazione lasciata allo sviluppatore
- Tipiche in UNIX (Solaris, Linux, Mac OS)

Esempio Pthreads

- Esempio somma dei primi N numeri
 - Per sempio per $N = 5$, $\text{Sum} = 1 + 2 + 3 + 4 + 5 = 15$
 - Si può delegare il calcolo ad uno o più thread
 - Abbiamo thread **sincroni** e **asincroni**
 - ▶ Nel caso sincrono il genitore aspetta i figli
 - ▶ Nel caso asincrono il genitore continua subito

Esempio Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```

Esempio Pthreads

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Codice Pthreads per il join di 10 Thread

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```


Pthreads

Per creare thread aggiuntionali relativi ad uno stesso processo, Posix prevede la funzione:

```
#include <pthread.h>
```

```
int pthread_create ( pthread_t *tid, const pthread_attr_t *attr,  
                    void * ( *start_func) (void *), void *arg );
```

- se la chiamata ha successo, *tid* punta al thread ID;
- *attr* permette di specificare gli attributi del thread (se *attr* = NULL, gli attributi sono quelli di default);
- *start_func* è l'indirizzo della funzione di avvio;
- *arg* è l'indirizzo dell'argomento accettato dalla funzione di avvio;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di <sys/errno.h> – in caso di errore.

Pthreads

```
typedef void (*thread_start)(void *);
```

```
int pthread_create(pthread_t      *tid,  
                  const pthread_attr_t *attributes  
                  thread_start    start,  
                  void             *argument);
```

- Restituisce 0 se OK, un codice d'errore altrimenti
- tid = argomento di ritorno, conterrà il tid del nuovo thread
- attributes = attributi del thread (vedere dopo)
- start = indirizzo della funzione da cui partire
- argument = l'argomento passato alla funzione start

Pthreads

```
void pthread_exit(void *status);
```

- termina il thread corrente, con valore di uscita status
- altri thread possono raccogliere il valore di uscita usando `pthread_join` (vedere slide successiva)
- fare attenzione che i dati puntati da ret sopravvivano alla terminazione del thread!
 - status non deve puntare allo stack (no variabili locali)
 - Ok uso di variabili globali o allocate dinamicamente

Pthreads

Un thread può attendere per la terminazione di un altro thread relativo allo stesso processo:

```
#include <pthread.h>
```

```
int pthread_join ( pthread_t *tid, void **status );
```

- *tid* è l'ID del thread del quale si vuole attendere la terminazione;
- *status* punta al valore restituito dal thread per cui si è atteso, indicante il suo stato di terminazione (se *status* = NULL, tale stato non viene restituito);
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```


Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```

Pthreads

```
/* crea il 1mo thread */
if ((en = pthread_create(&tid1, NULL, start_func, msg1) != 0))
    errno=en, perror("pthread_create"), exit(1);

/* crea il 2ndo thread */
if ((en = pthread_create(&tid2, NULL, start_func, msg2) != 0))
    errno=en, perror("pthread_create"), exit(2);

/* attende per il 1mo */
if ((en = pthread_join(tid1, NULL) != 0))
    errno=en, perror("pthread_join"), exit(1);

/* attende per il 2ndo */
if ((en = pthread_join(tid2, NULL) != 0))
    errno=en, perror("pthread_join"), exit(2);

return 0;
}
```

Pthreads

```
typedef struct foo{  
    int a;  
    int b;  
} myfoo;
```

```
myfoo test; // Variabile GLOBALE
```

```
void stampa(char *st, struct foo *test){  
    printf("%s: tid=%d a=%d b=%d\n", st, pthread_self(), test->a, test->b);  
}
```

```
void *fun1(void *arg){  
    myfoo test2 = {1,2}; // Variabile LOCALE  
    printf("%s %d\n", arg, pthread_self());  
    stampa(arg, &test2);  
    pthread_exit((void *)&test2);  
}
```


Pthreads

```
void *fun2(void *arg){
    test.a = 3;
    test.b = 4; // Variabile GLOBALE
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, &test);
    pthread_exit((void *)&test);
}
```

```
void *fun3(void *arg){
    myfoo *test3;
    test3=malloc(sizeof(struct foo)); // Variabile allocata dinamicamente
    test3->a = 5;
    test3->b = 6;
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, test3);
    pthread_exit((void *)test3); //c
}
```

Pthreads

```
int main(void){
    char st[100];
    pthread_t tid1;
    pthread_t tid2;
    pthread_t tid3;

    myfoo *b; // PUNTATORE alla struttura (non allocata)

    pthread_create(&tid1, NULL, fun1, "Thread 1"); // Locale
    pthread_join(tid1, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid2, NULL, fun2, "Thread 2"); // Globale
    pthread_join(tid2, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid3, NULL, fun3, "Thread 3"); // Dinamica
    pthread_join(tid3, (void *)&b);
    stampa("Master ", b);
}
```

Pthreads

Thread 1: 1077283760

// Locale

Thread 1: a=1 b=2

Master : a=1075156600 b=1077281896

Thread 2: 1077283760

// Globale

Thread 2: a=3 b=4

Master : a=3 b=4

Thread 3: 1077283760

// Dinamica

Thread 3: a=5 b=6

Master : a=5 b=6